Lab2: convolution

September 21, 2026

1 Introduction

In image processing, convolution filtering is a very important operation. A convolution is the operator denoted by $\star$ and defined as follows:

$$f \star g(x) = \int f(x-y)g(y)dy \tag{1}$$

where f and g are two integrable real functions. For functions in $\mathbb{R}^2$, the definition becomes:

$$f \star g(x', y') = \int\int f(x'-x, y'-y)g(x,y)dxdy \tag{2}$$

In the discrete space where images reside, the formula is written:

$$f \star g(k,l) = \sum_i \sum_j f(k-i, l-j)g(i,j) \tag{3}$$

We assume that our functions have a bounded support (i.e., a bounded domain of definition). Thus, the indices i and j vary within a finite interval. More precisely, assuming f is defined on $[1,N]\times[1,P]$ and g is defined on $[-\frac{n}{2},\frac{n}{2}]\times[-\frac{p}{2},\frac{p}{2}]$, the formula (3) becomes:

$$f \star g(k,l) = \sum_{\substack{1 \leq k-i \leq N \\ -\frac{n}{2} \leq i \leq \frac{n}{2}}} \sum_{\substack{1 \leq l-j \leq P \\ -\frac{p}{2} \leq j \leq \frac{p}{2}}} f(k-i, l-j)g(i,j) \tag{4}$$

This definition is more complex than the previous one. Indeed, convolution involves, among other things, translating one of the two functions. On an infinite support, the translation has no effect on the support. However, with a finite support, the domain is also translated. The consequence is therefore that the support of the convolved function differs from the supports of the convolved functions. In the example of equation 4, the support of $f \star g$ is $[1-\frac{n}{2}, N+\frac{n}{2}] \times [1-\frac{p}{2}, P+\frac{p}{2}]$. Figure 1 illustrates this: the calculation of the pixel in the bottom right corner is possible; its value is a priori non-zero because it depends on the value of a pixel in I (the one in gray), the others being zero.

2 Applications to images

Let I be an image of size $N \times P$, $I(x,y)$ is the luminance value of pixel (x,y). This image can be *convolved* with another image k, which is called the *convolution kernel*. The kernel k is generally small (typically 3×3, 5×5, or 7×7) and often represents a differential operator (but not necessarily); it is represented centered at 0.

For example, if we want to calculate the derivative at x of I ($\frac{\partial I}{\partial x}$), we can approximate by finite difference (first-order Taylor expansions) the operator:

$$\frac{\partial I(x,y)}{\partial x} \sim \frac{1}{2}(I(x+1,y) - I(x-1,y)) \tag{5}$$

which amounts to calculating $I \star k$ with:

$$k = \begin{pmatrix} 0 & 0 & 0 \\ -1/2 & 0 & 1/2 \\ 0 & 0 & 0 \end{pmatrix} \tag{6}$$

With other operators, we can smooth an image, extract contours, etc. Convolution can thus be seen as a weighted averaging operation on a neighborhood fixed by the convolution kernel (see Figure 1).

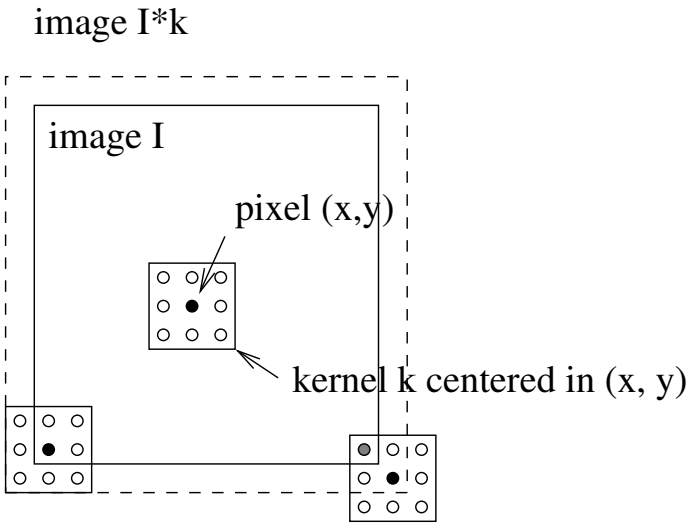


Figure 1: Convolution of I by k: $I \star k(x, y)$ is the weighted sum (by k) of the values of I at (x,y) and their neighbors. This also illustrates that the domain of $I \star k$ (dotted line) is the domain of I augmented by that of k. Beyond this, the values are zero.

3 Sequential algorithm

With the given definition of discrete convolution (equation (4)), we cannot calculate the convolution on the pixels at the edges of the image because it requires access to undefined pixels. However, calculations at the edges are possible (see Figure 1, bottom left and bottom right). To solve this problem, there are two methods:

- either we artificially enlarge the image size, in which case the new pixels have a value of zero;
- or we exclude the pixels at the edges of the image from the convolution calculation.

We choose here the second method because our goal is not to rigorously implement the convolution but to parallelize it.

To achieve long computation times and study the impact of MPI communications, the convolution operation is iterated a large number of times. Therefore, we want to implement the following operation:

$$I \star^n k = I \star \underbrace{k \star k \cdots \star k}_{n \text{ fois}} \tag{7}$$

Since many numerical applications (such as solving partial differential equations) require the repetition of a large number of convolutions, this choice is appropriate.

The final algorithm is rather simple :

1. Reading the command line (via the argv argument);
2. Loading the image (see the read_rasterfile function);
3. Repeat nbiter times the convolution of the image ("pointed to" by the r.data attribute of the variable r of type Raster) using one of the 5 filters:
   (a) memory allocation of an intermediate image buffer,
   (b) for each pixel (i, j) of the image (except the edges), do the following:

- the value of the pixel (i, j) in the buffer image is set to a linear combination (this is the convolution operation) of the pixel (i, j) and its 8 nearest neighbors in the image (r . data);

(c) copying the intermediate buffer into the image (r . data) and freeing the intermediate image buffer from memory;

4. Saving the image (see the save_rasterfile function).

The 5 different linear combinations represent the five possible filters (see the **switch** statement in the code). These operations are, in order: an averaging filter, another averaging filter (with more weight in the center), an edge detector (this is the Laplacian operator discretized by finite differences: such an operator gives a strong response on "peak" type edges, see Figure 2), another edge detector that detects "step" type edges (see Figure 2), and finally a median filter (non-linear: the pixels are sorted according to their brightness and the median value is retained).

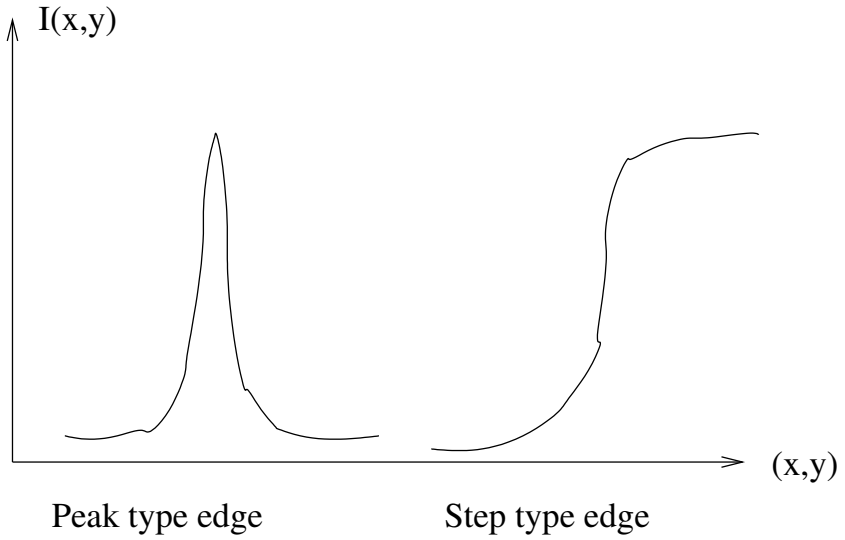


Figure 2: Examples of edge types.

4 MPI work description

1. In the convolution () function, why is an intermediate buffer necessary instead of performing the calculation directly on the image?
2. Which parts of the algorithm can be parallelized?
3. Given that the size of the convolution kernel k is 3×3 pixels, what is the theoretical operation count for processing one pixel of $I \star k$? What type of load balancing is relevant among the MPI processes?
4. What partitioning (data distribution between processes) is natural in this context?
5. What problem (at the edges of the image blocks) arises during the iterations of the convolution operation?
6. Implement a parallel algorithm with blocking MPI communications. Analyze the performance results.
7. Implement a second algorithm using non-blocking MPI communications in order to overlap communications with computations. Analyze the performance results.